

STUDIO:BY ANY MEDIA NECESSARY

DISTRACTION HOOK

Process Documentation | 7 Weeks

Misri Bhagat

M.A. Experience Design - Year 2

I started with an idea.

*I ended with something I
never thought I could make.*

7 weeks of ideas, experiments, failures, confusion and trying again. There were many moments when I felt, "This is too much," especially when I was working with things I had never done before. I started by knowing almost nothing about electronics, coding or 3D printing, but kept learning and trying. By the last day, I had a my working artefact in front of me and I felt proud of how far I had come. This is a documentation of that journey.



The studio began with an exploration of the Luddite Movement and its relationship with technology. During the Industrial Revolution, machines were introduced into textile production and started replacing the work of skilled workers. Many workers feared of losing their jobs and livelihoods because of these machines. In response, they destroyed machines as a form of protest.

While learning about the Luddites, I started thinking about what happens when technology enters an existing system and changes the way people work. I am interested in what happens to the people who are already part of a system when new technology changes it. This led me to think about a simple question: **When technology changes a system, who benefits and who gets left behind?**

I then started looking at this question through something much closer to everyday life. I looked at the shift from offline shopping to quick commerce. Earlier, buying groceries usually meant going to a nearby local shop, talking to the shopkeeper and buying what was needed. Now, with quick commerce platforms, the same things can be ordered through a phone and delivered within minutes. This change in shopping much faster and more convenient, but it also made me think about what happens to the local shops that were already part of this system.

I started noticing that the change was not only about convenience. It also had social, political and economic effects. Small retailers could be affected as people's shopping habits changed and more people started ordering through digital platforms. There was also a question of how large quick commerce companies and small local shops could compete fairly. Economically, things like price competition, changing retail practices and minimum order conditions for free delivery could also affect small businesses.

This brought me back to the Luddites. The Luddites had used the hammer to resist machines that they felt were threatening their livelihoods. I started asking myself, if technology is the machine of our time, then what could be the new hammer in the server room?

For me, the answer was collaboration. Instead of thinking about technology as something that has to replace the existing system, I started thinking about whether it could work with the people who are already part of that system. In the case of quick commerce, I imagined a model where local grocery stores could become part of the delivery network instead of being pushed out by it. Technology could then support the existing businesses rather than simply replacing them.

Critical Making: Conceptual and Material Studies in Technology and Social Life by Matt Ratto.

After exploring the Luddites, we moved towards Critical Making. We read Critical Making: Conceptual and Material Studies in Technology and Social Life by Matt Ratto. This made me start thinking about making not only as a way of producing something, but also as a way of understanding an idea through the process of making.

Along with the reading, we visited the Art and Science Lab in our college. There, I saw 3D printing and different kinds of technical making happening around me. For me, this was very new. I had never done 3D printing or worked with electronics before. I remember feeling very confused while looking at the machines and the process. I did not really understand what was happening there, and at that point I was quite sure that I would never choose 3D printing for my own work.

At that stage, I was also thinking about what kind of making I was comfortable with. Coming from a visual design background, working with electronics, machines and 3D printing felt very different from what I had done before. I was more used to working with graphic design and visual outcomes, so this kind of technical making felt unfamiliar to me.

Protocol: How Control Exists After Decentralization by Alexander Galloway.

After the Critical Making reading and the visit to the Art and Science Lab, we started reading book called Protocol: How Control Exists After Decentralization by Alexander Galloway. The book was divided among 14 people, and I have chosen the chapter **Tactical Media**.

When I started reading the chapter, I understood Tactical Media through the way people use technology and existing systems to question or challenge how those systems work. The chapter talks about different examples from digital media, including hackers, computer viruses, cyberfeminism and online protests that temporarily overload or block access to a website. What I found interesting was that these examples were not simply about rejecting technology. They were about finding ways to work within the same systems and use their own structures differently.

From Tactical Media, I started seeing a different way of thinking. Instead of simply breaking or rejecting the technology, people could use the same technology in a different way to question what it is doing.

At this point, I still had no idea what I wanted to do for my final project. I was mainly trying to understand how technology works within systems and how people can find different ways of using those systems.

The idea that stayed with me was that technology does not always have to be accepted exactly as it is given to us. The way we use it, respond to it and change our relationship with it can also be questioned.

A1 - From Exploration of Ideas to Final Idea Selection

After the earlier explorations around Luddite Movement, Critical Making and Protocol I started noticing for a protocol that I could observe in everyday life and work. I had a few different directions in mind, and I was trying to understand which one I was more interested in exploring.

One of the idea I had was **quick commerce**. I had already looked at the shift from local shops to online delivery and the effect this change could have on small retailers. I was interested in how a new digital system can change an existing way of doing things. I had also started thinking about how technology could work along with local shops instead of replacing them.

Another direction I thought about was the **shift from handwriting to digital writing**. I started noticing how children today are becoming more used to typing and writing digitally instead of writing by hand. I was interested in how this change is slowly becoming a normal part of everyday life. It made me think about how easily we adopt a new system when it becomes more convenient, and what might happen to the older habits that we leave behind.

I also had an idea around **invisible influence**. I started thinking about how digital platforms can quietly shape our everyday choices. We often feel that we are making our own decisions, but the things we see on platforms, the recommendations we receive and the way information is presented can influence what we choose.

Along with this, I was also thinking about the **influence of the people around us**. The people we live with, work with or spend time with can also affect the decisions we make without us always noticing it. I was interested in this idea of how our everyday choices are shaped by things around us, even when we feel that the choice is completely our own.

I had these different directions, but I realised that I was most interested in something that I experienced very directly in my own everyday life. I started looking more closely at my relationship with my phone. I noticed how a notification could interrupt whatever I was doing. I could be working, studying or doing something else, and a small sound from my phone could immediately make me look at it.

This felt different from the other ideas because I could clearly observe it happening around me. I started paying attention to what happens in that small moment: I am doing something, the phone gives me a notification, I notice it, and my attention moves towards the phone.

I became interested in this shift of attention rather than the larger idea of phone usage. I was not trying to say that phones are bad or that people should stop using them. I wanted to understand this small everyday behaviour and what happens when a notification pulls us away from what we were already doing.

This became the direction I decided to take forward. I chose to focus on the protocol of notification and attention shift, which eventually led me towards the concept of Distraction Hook.

A2 - From Idea to Prototype

Starting with the idea

After deciding to focus on the moment when a phone notification comes and our attention shifts towards it, I started thinking about how I could create that experience physically. I did not want to simply explain that notifications distract us. I wanted to create an experience where the notification itself could be felt through the object.

Choosing the materials - ESP32, microphone, proximity sensor, LEDs, buzzer, etc.

I started by thinking about what I would need. I decided to use an ESP32, a microphone, a proximity sensor, lights and a buzzer. At that time, I did not even know what an ESP32 was or how a microphone could detect a notification sound. I also had no knowledge of coding or electronics. I had never worked with jumper wires before, and I did not know how different components connected to each other or how electricity went through them.

I mostly had a direction in my head, so I first purchased the materials and then started figuring out how to use them. My peers helped me understand what I needed to install, how to connect the components and how to start with the coding. I also used ChatGPT to understand the technical steps and to help me with the coding. I was learning these things while making the prototype, rather than knowing them before I started.

I decided to test each component separately first. I started with the ESP32 and checked whether the board itself was working. Once that was working, I connected the INMP441 microphone and tested whether it could detect sound. I checked the sound values through the serial monitor and tried different sounds to understand what the microphone was detecting.

After the microphone, I connected LED Lights and then I connected ESP32 to the proximity sensor. I tested whether it could detect the distance between the phone and the object. Each of these tests had its own small piece of code. I am attaching the below.

ESP32 test

```
1  #include <driver/i2s.h>
2
3  #define I2S_WS   25
4  #define I2S_SCK  26
5  #define I2S_SD   33
6
7  void setup() {
8      Serial.begin(115200);
9
10     i2s_config_t i2s_config = {
11         .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
12         .sample_rate = 16000,
13         .bits_per_sample = I2S_BITS_PER_SAMPLE_32BIT,
14         .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,
15         .communication_format = I2S_COMM_FORMAT_I2S,
16         .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
17         .dma_buf_count = 8,
18         .dma_buf_len = 64,
19         .use_apll = false,
20         .tx_desc_auto_clear = false,
21         .fixed_mclk = 0
22     };
23
24     i2s_pin_config_t pin_config = {
25         .bck_io_num = I2S_SCK,
26         .ws_io_num = I2S_WS,
27         .data_out_num = I2S_PIN_NO_CHANGE,
28         .data_in_num = I2S_SD
29     };
30
31     i2s_driver_install(I2S_NUM_0, &i2s_config, 0, NULL);
32     i2s_set_pin(I2S_NUM_0, &pin_config);
33
34     Serial.println("INMP441 microphone started!");
35 }
36
37 void loop() {
38     int32_t samples[64];
39     size_t bytes_read;
40
41     i2s_read(
42         I2S_NUM_0,
43         samples,
44         sizeof(samples),
45         &bytes_read,
46         portMAX_DELAY
47     );
48
49     long sum = 0;
50
51     for (int i = 0; i < 64; i++) {
52         sum += abs(samples[i] >> 14);
53     }
54
55     long average = sum / 64;
56
57     Serial.println(average);
58
59     delay(50);
60 }
```

ESP32 test + Microphone

```
1  #include <driver/i2s.h>
2
3  #define I2S_PORT I2S_NUM_0
4
5  #define SCK_PIN 26
6  #define WS_PIN 25
7  #define SD_PIN 34
8
9  void setup() {
10     Serial.begin(115200);
11
12     i2s_config_t config = {
13         .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
14         .sample_rate = 16000,
15         .bits_per_sample = I2S_BITS_PER_SAMPLE_32BIT,
16         .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,
17         .communication_format = I2S_COMM_FORMAT_I2S,
18         .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
19         .dma_buf_count = 4,
20         .dma_buf_len = 256,
21         .use_apll = false,
22         .tx_desc_auto_clear = false,
23         .fixed_mclk = 0
24     };
25
26     i2s_pin_config_t pins = {
27         .bck_io_num = SCK_PIN,
28         .ws_io_num = WS_PIN,
29         .data_out_num = I2S_PIN_NO_CHANGE,
30         .data_in_num = SD_PIN
31     };
32
33     i2s_driver_install(I2S_PORT, &config, 0, NULL);
34     i2s_set_pin(I2S_PORT, &pins);
35     i2s_zero_dma_buffer(I2S_PORT);
36
37     Serial.println("INMP441 TEST STARTED");
38 }
39
40 void loop() {
41     int32_t samples[256];
42     size_t bytesRead = 0;
43
44     i2s_read(
45         I2S_PORT,
46         samples,
47         sizeof(samples),
48         &bytesRead,
49         portMAX_DELAY
50     );
51
52     long long total = 0;
53     int count = bytesRead / sizeof(int32_t);
54
55     for (int i = 0; i < count; i++) {
56         int32_t sample = samples[i] >> 14;
57         total += abs(sample);
58     }
59
60     if (count > 0) {
61         long soundLevel = total / count;
62
63         Serial.print("Sound: ");
64         Serial.println(soundLevel);
65     }
66
67     delay(500);
68 }
```

ESP32 test + Microphone + LED Light

```
1  #include <driver/i2s.h>
2
3  // =====
4  // MICROPHONE
5  // =====
6  #define I2S_PORT I2S_NUM_0
7
8  #define SCK_PIN 26
9  #define WS_PIN 25
10 #define SD_PIN 34
11
12 // =====
13 // 4 LEDs
14 // =====
15 #define LED1 16
16 #define LED2 17
17 #define LED3 18
18 #define LED4 19
19
20 // =====
21 // SETTINGS
22 // =====
23
24 // Change this number if needed
25 #define SOUND_THRESHOLD 300
26
27 // LEDs stay flickering for 3 seconds
28 #define FLICKER_TIME 3000
29
30 // LED flicker speed
31 #define FLICKER_SPEED 100
32
33 // =====
34 // VARIABLES
35 // =====
36 bool flickering = false;
37
38 unsigned long flickerStartTime = 0;
39 unsigned long lastFlicker = 0;
40
41 bool ledState = false;
42
43 // =====
44 // SETUP
45 // =====
46 void setup() {
47     Serial.begin(115200);
48
49     // LEDs
50     pinMode(LED1, OUTPUT);
51     pinMode(LED2, OUTPUT);
52     pinMode(LED3, OUTPUT);
53     pinMode(LED4, OUTPUT);
54
55     digitalWrite(LED1, LOW);
56     digitalWrite(LED2, LOW);
57     digitalWrite(LED3, LOW);
58     digitalWrite(LED4, LOW);
59
60     // =====
61     // MICROPHONE SETUP
62     // =====
63
64     i2s_config_t config = {
65         .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
66         .sample_rate = 16000,
67         .bits_per_sample = I2S_BITS_PER_SAMPLE_32BIT,
68         .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,
69         .communication_format = I2S_COMM_FORMAT_I2S,
70         .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
71         .dma_buf_count = 4,
72         .dma_buf_len = 256,
73         .use_apll = false,
74         .tx_desc_auto_clear = false,
75         .fixed_mclk = 0
76     };
77
78     i2s_pin_config_t pins = {
79         .bck_io_num = SCK_PIN,
80         .ws_io_num = WS_PIN,
81         .data_out_num = I2S_PIN_NO_CHANGE,
82         .data_in_num = SD_PIN
83     };
84
85     i2s_driver_install(I2S_PORT, &config, 0, NULL);
86     i2s_set_pin(I2S_PORT, &pins);
87     i2s_zero_dma_buffer(I2S_PORT);
88
89     Serial.println("=====");
90     Serial.println("MICROPHONE + LED TEST");
91     Serial.println("=====");
92     Serial.println("Waiting for sound...");
93 }
94
95 // =====
96 // TURN ALL LEDs OFF
97 // =====
98 void turnLEDsOff() {
99     digitalWrite(LED1, LOW);
100     digitalWrite(LED2, LOW);
101     digitalWrite(LED3, LOW);
102     digitalWrite(LED4, LOW);
103 }
104
105 // =====
106 // TURN ALL LEDs ON
107 // =====
108 void turnLEDsOn() {
109     digitalWrite(LED1, HIGH);
110     digitalWrite(LED2, HIGH);
111     digitalWrite(LED3, HIGH);
112     digitalWrite(LED4, HIGH);
113 }
114
115 }
```

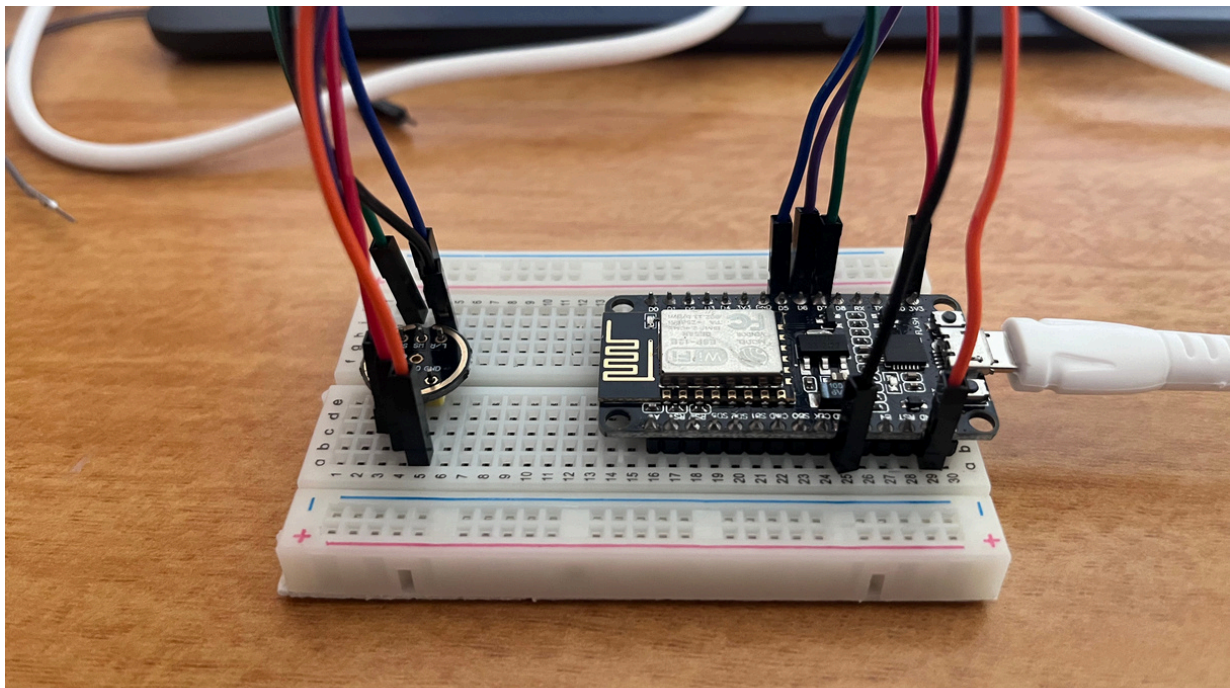
```

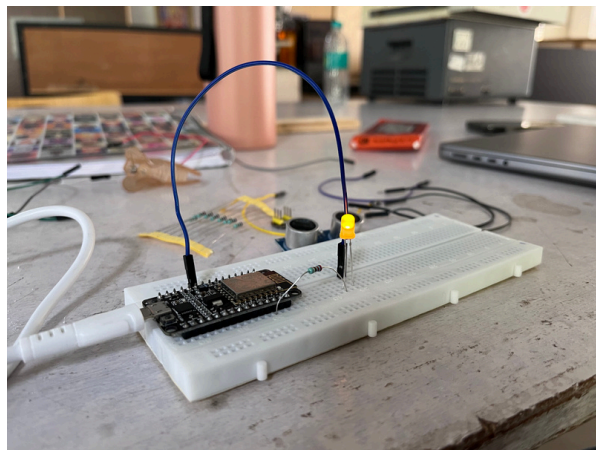
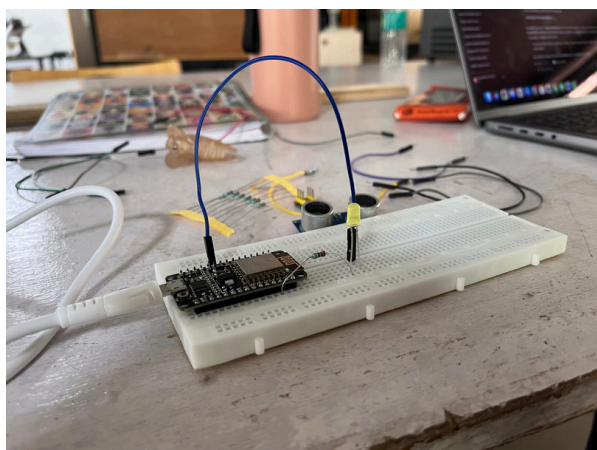
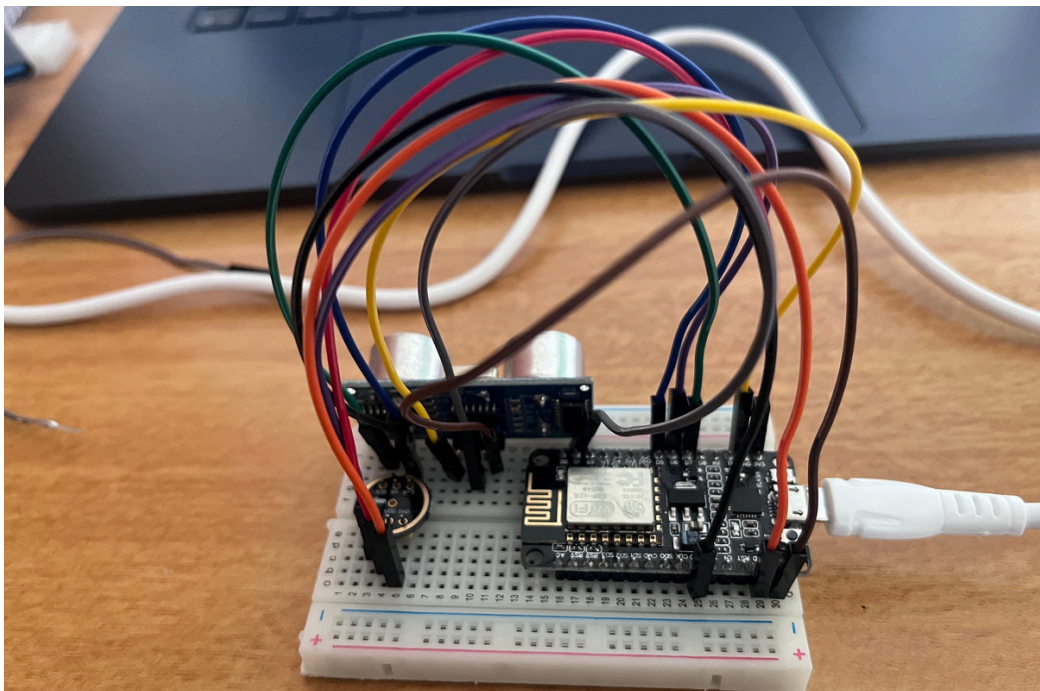
122
123 // =====
124 // MAIN LOOP
125 // =====
126 void loop() {
127
128 // =====
129 // READ MICROPHONE
130 // =====
131
132 int32_t samples[256];
133 size_t bytesRead = 0;
134
135 i2s_read(
136     I2S_PORT,
137     samples,
138     sizeof(samples),
139     &bytesRead,
140     portMAX_DELAY
141 );
142
143 int sampleCount = bytesRead / sizeof(int32_t);
144
145 if (sampleCount == 0) {
146     return;
147 }
148
149 long long total = 0;
150
151 for (int i = 0; i < sampleCount; i++) {
152     int32_t sample = samples[i] >> 14;
153
154     total += abs(sample);
155 }
156
157 long soundLevel = total / sampleCount;
158
159 // Show sound level
160 Serial.print("Sound: ");
161 Serial.println(soundLevel);
162
163 unsigned long currentTime = millis();
164
165 // =====
166 // START FLICKERING
167 // =====
168
169 if (!flickering && soundLevel >= SOUND_THRESHOLD) {
170     flickering = true;
171     flickerStartTime = currentTime;
172     lastFlicker = currentTime;
173     ledState = true;
174     turnLEDsOn();
175     Serial.println("!!! SOUND DETECTED - LEDs ON !!!");
176 }
177
178 // =====
179 // FLICKERING
180 // =====
181
182 if (flickering) {
183     // Check if 3 seconds are finished
184     if (currentTime - flickerStartTime >= FLICKER_TIME) {
185         flickering = false;
186         turnLEDsOff();
187         Serial.println("LEDs OFF - READY");
188     }
189
190     // Flicker LEDs
191     else if (currentTime - lastFlicker >= FLICKER_SPEED) {
192         lastFlicker = currentTime;
193         ledState = !ledState;
194         if (ledState) {
195             turnLEDsOn();
196         }
197         else {
198             turnLEDsOff();
199         }
200     }
201 }
202
203 delay(10);
204

```

ESP32 test + Proximity Sensor

```
1 #define TRIG_PIN 5
2 #define ECHO_PIN 4
3
4 #define MAX_DISTANCE 15.0
5
6 void setup() {
7   Serial.begin(115200);
8
9   pinMode(TRIG_PIN, OUTPUT);
10  pinMode(ECHO_PIN, INPUT);
11
12  digitalWrite(TRIG_PIN, LOW);
13
14  Serial.println("HC-SR04 PROXIMITY TEST");
15  Serial.println("-----");
16  Serial.println("Sensor test started...");
17 }
18
19 void loop() {
20
21  // Send ultrasonic pulse
22  digitalWrite(TRIG_PIN, LOW);
23  delayMicroseconds(2);
24
25  digitalWrite(TRIG_PIN, HIGH);
26  delayMicroseconds(10);
27
28  digitalWrite(TRIG_PIN, LOW);
29
30  // Read echo
31  long duration = pulseIn(ECHO_PIN, HIGH, 30000);
32
33  if (duration == 0) {
34
35    Serial.println("SENSOR STATUS: NOT WORKING / NO ECHO");
36
37  } else {
38
39    float distance = duration * 0.0343 / 2;
40
41    Serial.print("SENSOR STATUS: WORKING | ");
42    Serial.print("Distance: ");
43    Serial.print(distance);
44    Serial.print(" cm ~");
45
46    if (distance <= MAX_DISTANCE) {
47      Serial.println("PHONE WITHIN 15 CM");
48    } else {
49      Serial.println("PHONE BEYOND 15 CM");
50    }
51  }
52
53  delay(500);
54 }
```





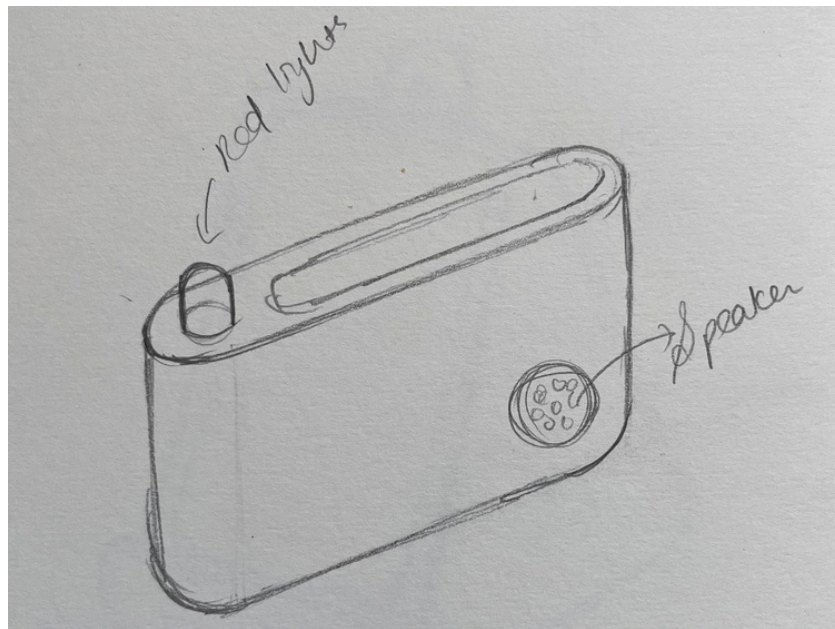
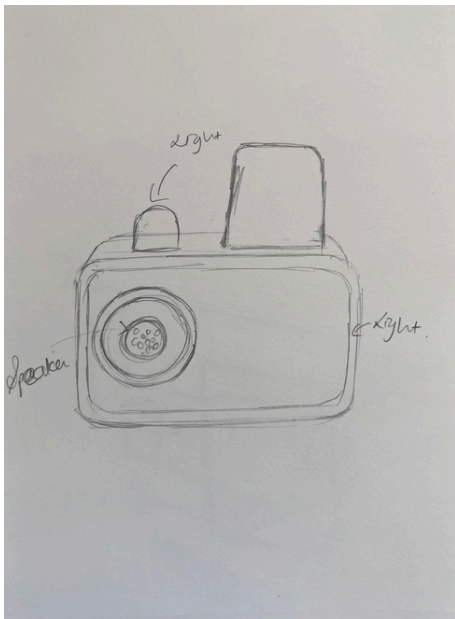
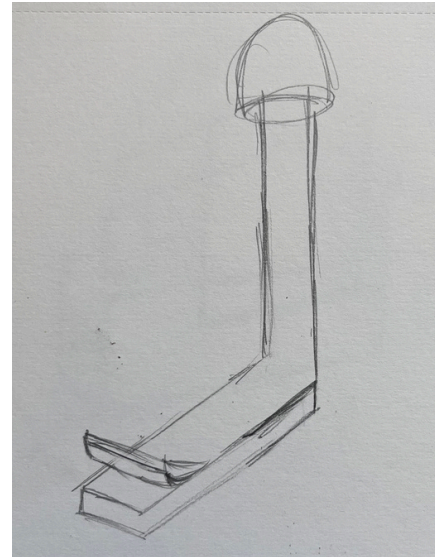
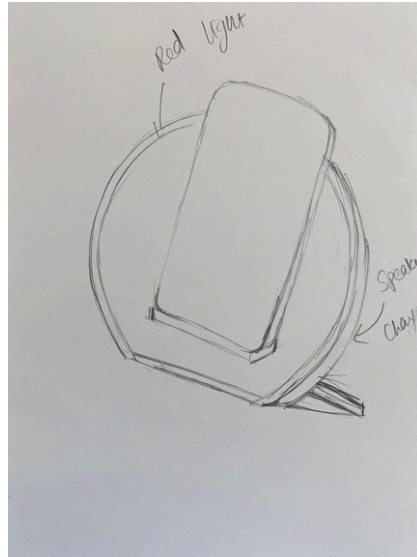
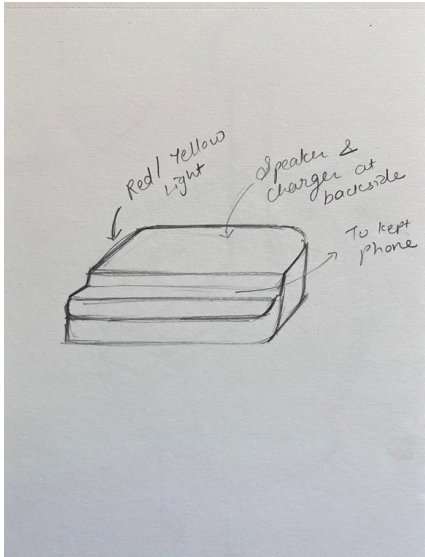
After testing the components separately, I started combining them. This is the time where I faced most of the difficulties. A code that worked for one component did not work when I added another component. Sometimes the microphone worked but the other parts did not. Sometimes the sensor worked but the complete code did not work. I had to keep changing the code, uploading it, testing it and trying again.

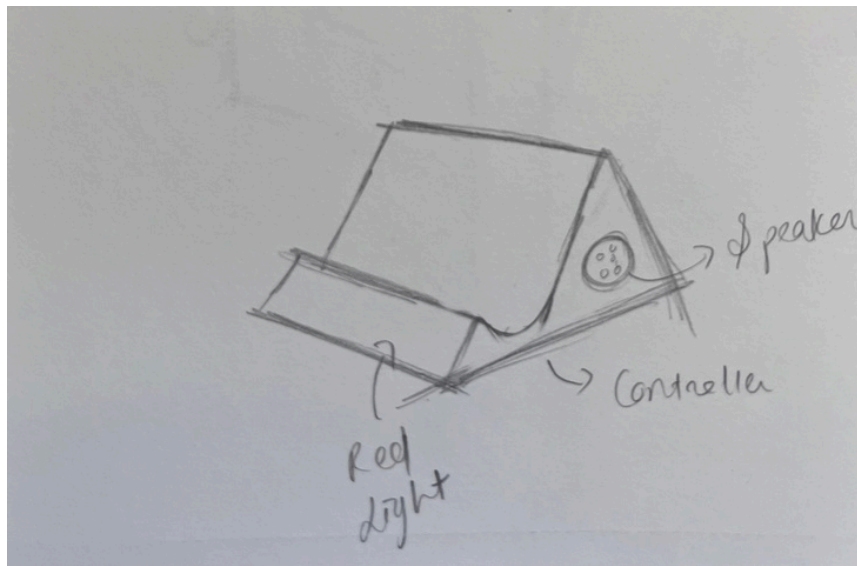
There were also many problems that were not related to the code like a jumper wire was not working. Sometimes a connection was loose. Sometimes I realized that I needed another material to make something work, so I had to go back to the shop and buy it.

At one point, I became very frustrated with the electronics and felt that I did not want to continue anymore. There were so many small things to figure out, and sometimes I would fix one thing and another thing would stop working. But I kept testing because I wanted to see the experience I had imagined actually happen.

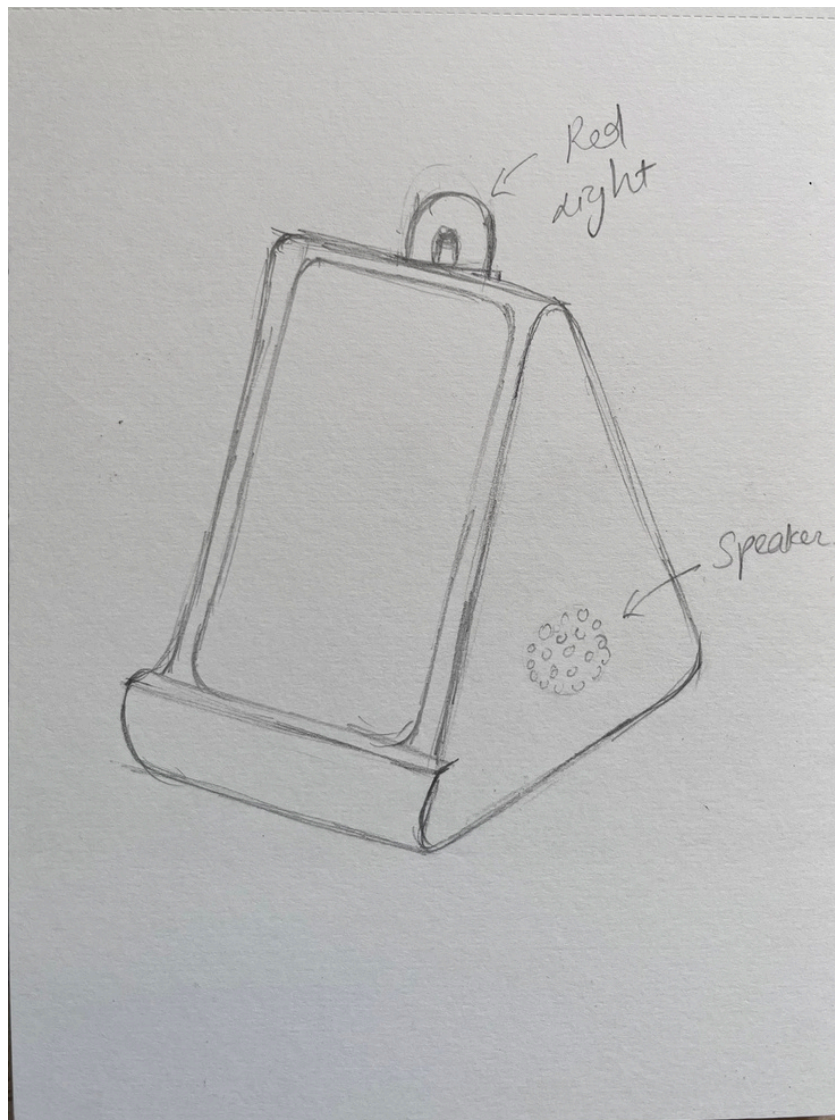
From Sketch to Form

I was also figuring out the form of the phone stand side by side. In the beginning, I did not know how my phone stand would look like. I made a few sketches and from those sketches, two forms were selected by Manuel. After that, I decided to make the stand using 3D printing.





DUMMY FORM

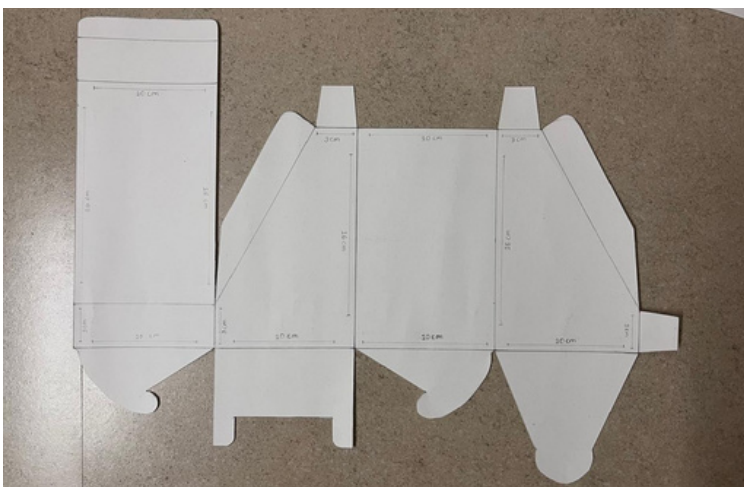
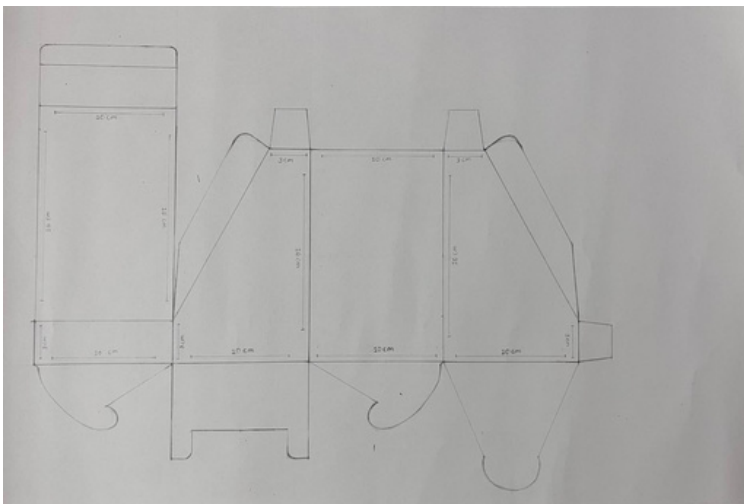


FINAL FORM - MOBILE STAND

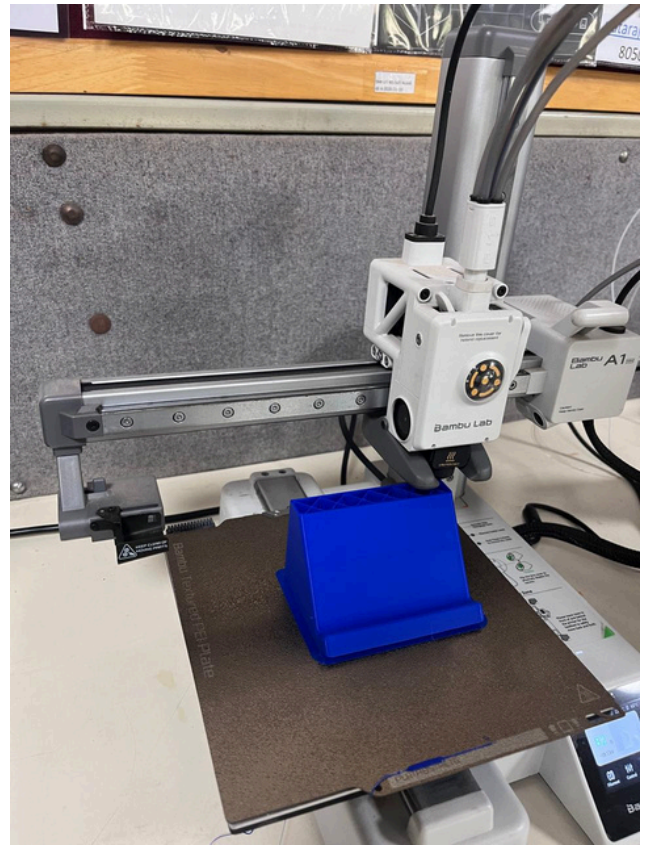
I had never done 3D printing before so I did not know how to create the 3D print file. I looked at online source like Printable to see if I could find something similar but I could not find the form which I needed. So I first started with paper form and than I started learning online how to make 3D model step by step using Fusion 360. I used ChatGPT to guide me through the process because I was completely new to making 3D files.

The first 3D print I have done helped me understand something that I had not thought about earlier. When I saw the printed form I got to know that I need hollow from inside to put all the components like electronic components and wires. I had made the form without thought of it.

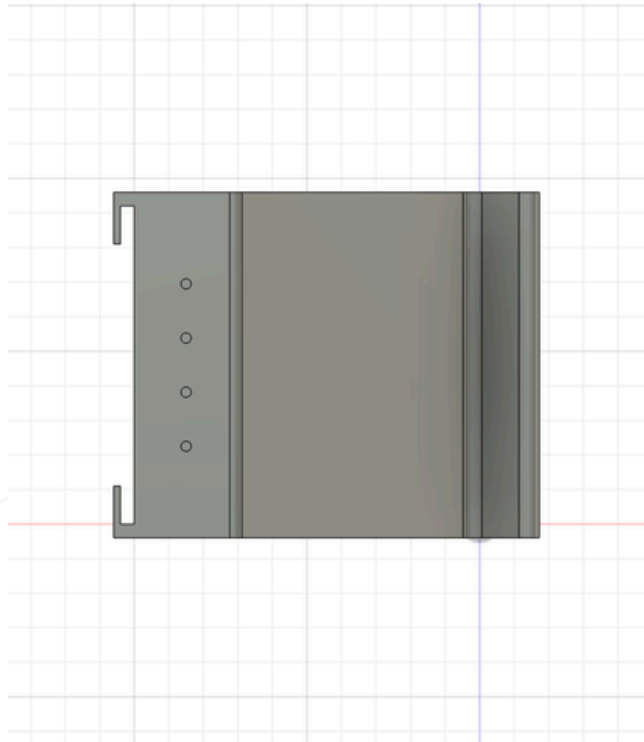
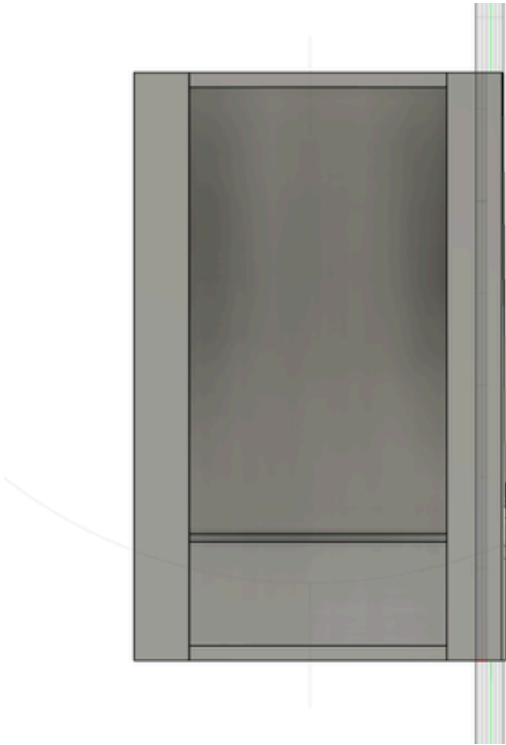
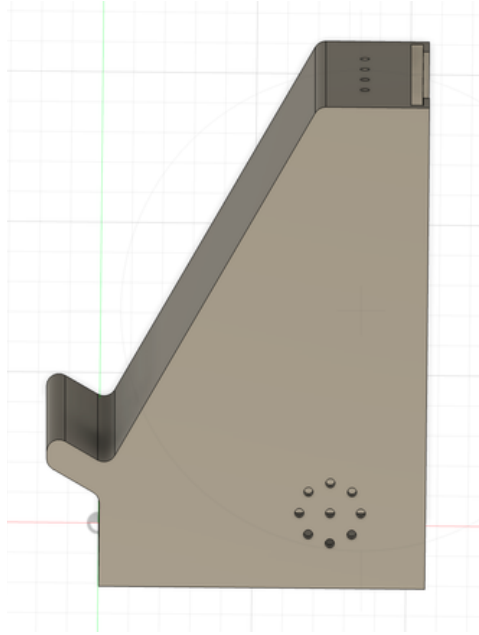
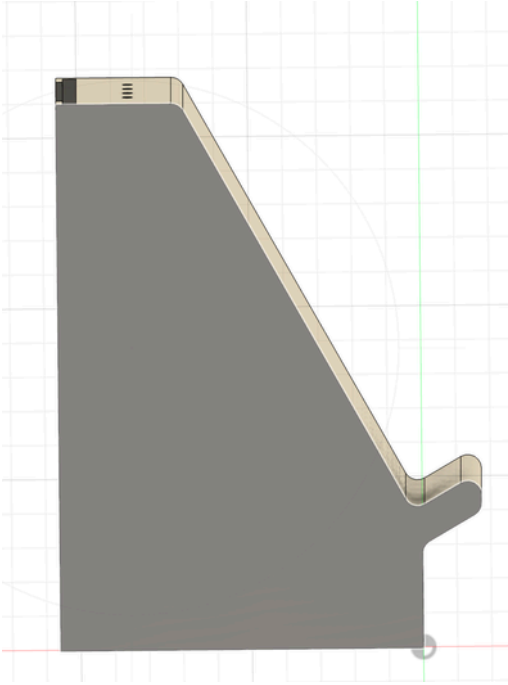
I then divided the stand into two parts. The front part which is first part is the main body, where the phone would sit, and the back part that is second part would close the object. I decided to make the back part sliding similar to how the back of a TV remote slides in. This allowed me to keep the electronics inside the object and still open it when I needed to access them.



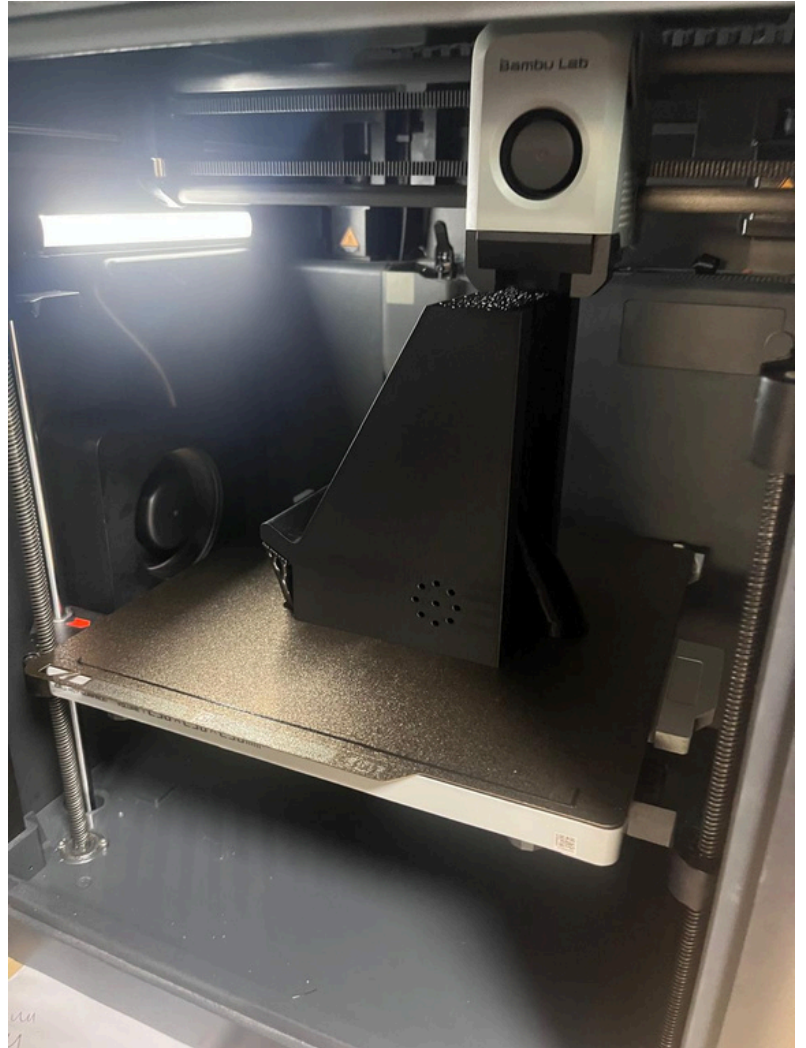
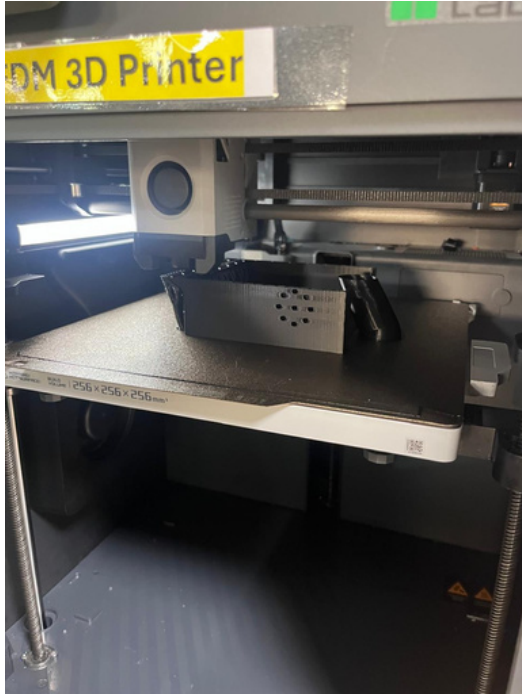
DUMMY FORM



FIRST 3D PRINTED DUMMY



FINAL 3D PRINTED ARTEFACT



First Prototype - Presentation

After connecting and testing the electronics and making the first 3D printed form, I made a dummy version of Distraction Hook for my first presentation. It was not final artefact but I wanted to show the idea to people and see people understood it or not.

I presented the prototype and explained what I was trying to communicate through it. During the presentation, I received feedback that the prototype was a little confusing by explainign and that the idea was not clear enough through the object itself.

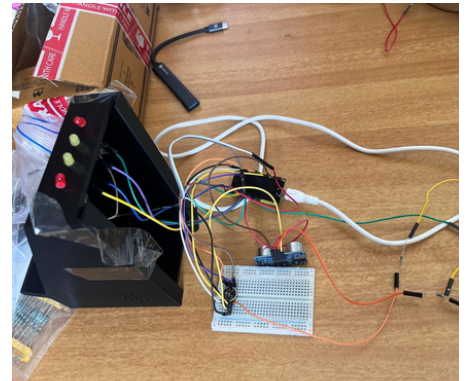
Feedback I Received

I had a though to keep the sensor inside the phone stand but I got to know that it would not work properly if it was covered by the 3D printed body. The sensor needed to be exposed outside so that it could detect the distance of the phone.

I got questions like:

- What prototype telling you?
- Is this design intervention or artefact?
- Why did you make this?
- How are you showing distraction through the object?
- Are you trying to make the moment of distraction more annoying?





A3 - From Prototype to Final Artefact

Changes I made after first presentation

After the first presentation feedback I made some changes to the phone stand. I realised that the proximity sensor could not be kept inside the stand because it needed to be outside to detect the phone. So, I made an opening in the stand and placed the sensor there. This was a small change but it was important for the phone stand to work properly.



Final Coding & Testing

After making the changes in the phone stand, I worked on the final code and tested all the components together. I checked if the sensor could detect when the phone was close, if the microphone could detect the notification sound, and if the lights and buzzer responded at the right time.

I also test what happened when I moved the phone away. The lights and buzzer needed to stop when the phone moved away from the stand.

I keep on testing these steps until the complete sequence was working properly.

```
1  #include <driver/i2s.h>
2
3  // =====
4  // MICROPHONE
5  // =====
6  #define I2S_PORT I2S_NUM_0
7
8  #define SCK_PIN 26
9  #define WS_PIN 25
10 #define SD_PIN 34
11
12 // =====
13 // 4 LEDs
14 // =====
15 #define LED1 16
16 #define LED2 17
17 #define LED3 18
18 #define LED4 19
19
20 // =====
21 // HC-SR04
22 // =====
23 #define TRIG_PIN 5
24 #define ECHO_PIN 4
25
26 // Alarm trigger
27 #define TRIGGER_DISTANCE 15.0
28
29 // Alarm release
30 #define RELEASE_DISTANCE 20.0
31
32 // =====
33 // BUZZER
34 // =====
35 #define BUZZER_PIN 23
36
37 // =====
38 // SOUND
39 // =====
40 #define SOUND_THRESHOLD 500
41
42 // =====
43 // LED FLICKER
44 // =====
45 #define FLICKER_SPEED 100
46
47 // =====
48 // PROXIMITY READ INTERVAL
49 // =====
50 #define PROXIMITY_INTERVAL 100
51
52 // =====
53 // VARIABLES
54 // =====
55 bool alarmActive = false;
56 bool ledState = false;
57
58 unsigned long lastFlicker = 0;
59 unsigned long lastProximityRead = 0;
60
61 float currentDistance = -1.0;
62 bool proximityWorking = false;
63
64
65 // =====
66 // LEDs OFF
67 // =====
68 void turnLEDsOff() {
69     digitalWrite(LED1, LOW);
70     digitalWrite(LED2, LOW);
71     digitalWrite(LED3, LOW);
72     digitalWrite(LED4, LOW);
73 }
74
75
76
```

```

76 // =====
77 // LEDs ON
78 // =====
79 void turnLEDsOn() {
80     digitalWrite(LED1, HIGH);
81     digitalWrite(LED2, HIGH);
82     digitalWrite(LED3, HIGH);
83     digitalWrite(LED4, HIGH);
84 }
85
86
87 // =====
88 // READ PROXIMITY
89 // =====
90 void readProximity() {
91
92     digitalWrite(TRIG_PIN, LOW);
93     delayMicroseconds(2);
94
95     digitalWrite(TRIG_PIN, HIGH);
96     delayMicroseconds(10);
97
98     digitalWrite(TRIG_PIN, LOW);
99
100     // 5 ms timeout
101     unsigned long duration = pulseIn(ECHO_PIN, HIGH, 5000);
102
103     if (duration == 0) {
104
105         proximityWorking = false;
106         currentDistance = -1.0;
107
108     } else {
109
110         proximityWorking = true;
111         currentDistance = duration * 0.0343 / 2.0;
112     }
113 }
114
115
116 // =====
117 // START ALARM
118 // =====
119 void startAlarm() {
120
121     alarmActive = true;
122
123     digitalWrite(BUZZER_PIN, HIGH);
124
125     ledState = true;
126     turnLEDsOn();
127
128     lastFlicker = millis();
129 }
130
131
132 // =====
133 // STOP ALARM
134 // =====
135 void stopAlarm() {
136
137     alarmActive = false;
138
139     digitalWrite(BUZZER_PIN, LOW);
140
141     ledState = false;
142     turnLEDsOff();
143 }
144
145
146 // =====
147 // SETUP
148 // =====
149 void setup() {
150
151     Serial.begin(115200);
152
153     // =====
154     // LED SETUP
155     // =====
156     pinMode(LED1, OUTPUT);
157     pinMode(LED2, OUTPUT);
158     pinMode(LED3, OUTPUT);
159     pinMode(LED4, OUTPUT);
160
161     turnLEDsOff();

```

```

162
163
164 // =====
165 // HC-SR04 SETUP
166 // =====
167 pinMode(TRIG_PIN, OUTPUT);
168 pinMode(ECHO_PIN, INPUT);
169
170 digitalWrite(TRIG_PIN, LOW);
171
172
173 // =====
174 // BUZZER SETUP
175 // =====
176 pinMode(BUZZER_PIN, OUTPUT);
177 digitalWrite(BUZZER_PIN, LOW);
178
179
180 // =====
181 // MICROPHONE SETUP
182 // =====
183 i2s_config_t config = {
184     .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
185     .sample_rate = 16000,
186     .bits_per_sample = I2S_BITS_PER_SAMPLE_32BIT,
187     .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,
188     .communication_format = I2S_COMM_FORMAT_I2S,
189     .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
190     .dma_buf_count = 4,
191     .dma_buf_len = 256,
192     .use_apll = false,
193     .tx_desc_auto_clear = false,
194     .fixed_mclk = 0
195 };
196
197 i2s_pin_config_t pins = {
198     .bck_io_num = SCK_PIN,
199     .ws_io_num = WS_PIN,
200     .data_out_num = I2S_PIN_NO_CHANGE,
201     .data_in_num = SD_PIN
202 };
203

```

```

204 i2s_driver_install(I2S_PORT, &config, 0, NULL);
205 i2s_set_pin(I2S_PORT, &pins);
206 i2s_zero_dma_buffer(I2S_PORT);
207
208 Serial.println("SYSTEM READY");
209 }
210
211
212 // =====
213 // MAIN LOOP
214 // =====
215 void loop() {
216
217     // =====
218     // READ MICROPHONE
219     // =====
220     int32_t samples[256];
221     size_t bytesRead = 0;
222
223     i2s_read(
224         I2S_PORT,
225         samples,
226         sizeof(samples),
227         &bytesRead,
228         portMAX_DELAY
229     );
230
231     int sampleCount = bytesRead / sizeof(int32_t);
232
233     if (sampleCount == 0) {
234         return;
235     }
236
237
238     long long total = 0;
239
240     for (int i = 0; i < sampleCount; i++) {
241
242         int32_t sample = samples[i] >> 14;
243
244         total += abs(sample);
245     }
246
247     long soundLevel = total / sampleCount;
248
249
250     // =====
251     // TIME
252     // =====
253     unsigned long currentTime = millis();
254

```

```

255
256 // =====
257 // READ PROXIMITY
258 // =====
259 if (currentTime - lastProximityRead >= PROXIMITY_INTERVAL) {
260
261     lastProximityRead = currentTime;
262
263     readProximity();
264 }
265
266
267 // =====
268 // TRIGGER CONDITION
269 // =====
270 // Alarm starts ONLY if:
271 //
272 // Proximity < 15 cm
273 // AND
274 // Sound > 1000
275 //
276 // This is checked ONLY when alarm is OFF.
277 // =====
278
279 if (!alarmActive) {
280
281     if (
282         proximityWorking &&
283         currentDistance < TRIGGER_DISTANCE &&
284         soundLevel > SOUND_THRESHOLD
285     ) {
286
287         startAlarm();
288     }
289 }
290
291
292 // =====
293 // RELEASE CONDITION
294 // =====
295 // Once alarm is ON:
296 //
297 // Ignore sound level completely.
298 //
299 // Keep alarm ON until:
300 //
301 // Proximity > 20 cm
302 // =====
303
304 if (alarmActive) {
305
306     if (
307         proximityWorking &&
308         currentDistance > RELEASE_DISTANCE
309     ) {
310
311         stopAlarm();
312     }
313 }
314
315
316 // =====
317 // LED FLICKERING
318 // =====
319 if (alarmActive) {
320
321     if (currentTime - lastFlicker >= FLICKER_SPEED) {
322
323         lastFlicker = currentTime;
324
325         ledState = !ledState;
326
327         if (ledState) {
328             turnLEDsOn();
329         } else {
330             turnLEDsOff();
331         }
332     }
333 }
334
335
336 // =====
337 // ONE-LINE DEBUG
338 // =====
339 Serial.print("Sound:");
340 Serial.print(soundLevel);
341
342 Serial.print(" | Proximity:");
343
344 if (proximityWorking) {
345     Serial.print(currentDistance, 1);
346     Serial.print("cm");
347 } else {

```

```

348     Serial.print("NO-ECHO");
349 }
350
351 Serial.print(" | Alarm:");
352 Serial.print(alarmActive ? "ON" : "OFF");
353
354 Serial.print(" | LEDs:");
355 Serial.print(alarmActive ? "FLICKER" : "OFF");
356
357 Serial.print(" | Buzzer:");
358 Serial.println(alarmActive ? "ON" : "OFF");
359
360 delay(10);
361 }
362

```

Once the testing was done I moved from jumper wires to soldering to secure the connections. The jumper wires were useful while testing while I needed stronger and more stable connections for the final artefact. Soldering was new for me so I had to do it carefully and keep checking the connections while working.

After soldering, I started putting all the components inside the final 3D printed body. I had to make sure that the wires and other components could fit properly inside the stand and also working properly.

After everything was placed, I tested the artefact again to make sure that nothing stopped working during the assembly.

USER JOURNEY OF DISTRACTION HOOK

1. **Approach** - User comes to the artefact with their phone.
2. **Place Phone** - User places the phone on the stand.
3. **Start Working** - User begins studying or working.
4. **Notification Arrives** - A notification comes on the phone.
5. **Distraction Hook responds** - The microphone detects the notification sound, and the lights and buzzer respond.
6. **Attention shifts** - The user notices the lights and sound and becomes aware of the notification.
7. **Check the notification** - The user checks the phone notification.
8. **Response stops** - The lights and buzzer stop, and the experience ends.

Final Presentation

I presented Distraction Hook and showed how the artefact works. I explained the idea behind it and demonstrated the experience of placing the phone on the stand, receiving a notification, and seeing the lights and buzzer respond.

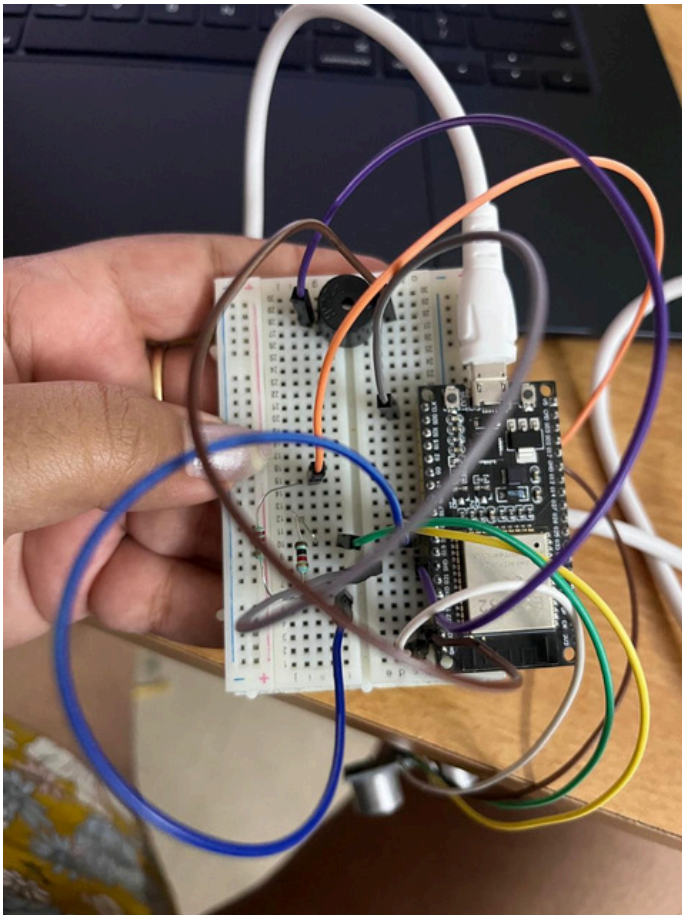
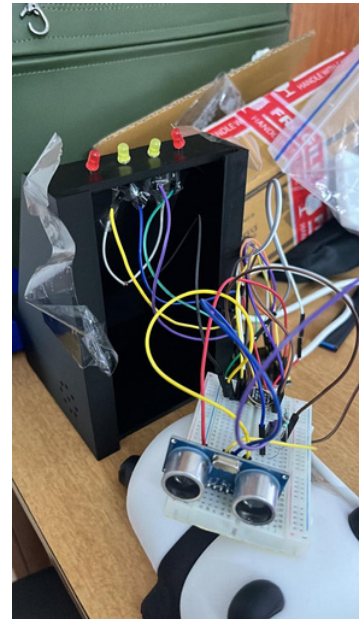
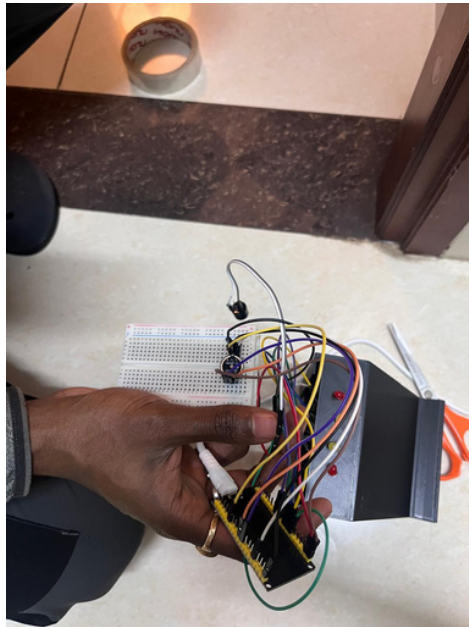
This presentation was also a chance for me to show the final working artefact after all the testing, changes and assembly I had done.



Watch the Final Working Artefact Video

<https://vimeo.com/1226021478?share=copy>

PROCESS PHOTOS



Reflection on my Learning and Experience

When I started this studio, I honestly had no idea how I was going to make something like this. I had never worked with electronics before. I did not even know what Arduino, ESP32 or Raspberry Pi were. The idea of working with wires, sensors and coding felt very difficult and confusing to me.

In the beginning, even understanding how one small component worked took a lot of time. Sometimes I would connect something and it would not work, and I would not even know what was wrong. I had to keep checking the wires, the code and the connections and try again.

There were many times when I felt frustrated. I realised that working with electronics needs a lot of patience. Things do not always work on the first try, and sometimes even a very small connection can stop the whole thing from working. I slowly learned to not get frustrated immediately and instead check what could be wrong and try again.

I learned that I do not have to know everything before I start. I started this project without knowing electronics, coding or 3D printing, but I learned these things while making the project.

The biggest thing I am taking from this studio is the confidence that I can learn something that initially feels very new to me. At the beginning, I could not imagine myself making an electronic prototype. By the end of the studio, I had a tangible, working artefact which made by me in front of me that I had built through all those small steps.

I am genuinely happy with what I made. More than just the final artefact, I am happy that I did not give up when the process became difficult. This studio taught me to be more patient, to keep trying when things fail, and to trust the process of learning by doing.

I started the studio not knowing about electronics and ended it with a working object made using electronics, coding and 3D printing. For me, that is probably the biggest learning from these seven weeks.

THANK YOU